

LAPORAN PRAKTIKUM

Modul 3: Jaringan Nirkabel dan Komunikasi HTTP



Oleh:

NAMA : Muhammad Hayya Faza Habibilah

NPM : 25782104

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

PRAKTIKUM

1. NodeMCU sebagai *Web Server* (Menyajikan HTML)

Skenario: Anda merakit NodeMCU sebagai server pintar. Modul ini membaca sensor DHT dan menampilkan suhunya di sebuah halaman web (UI), sekaligus menyediakan tombol di web tersebut untuk menyalakan/mematikan lampu LED.

Langkah Kerja Rangkaian:

1. Menghubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2) . (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Menghubungkan anoda LED ke pin D6 dan katoda ke resistor 220 Ohm dan terhubung ke GND

Langkah Kerja Pemrograman:

```
praktik3_5_1.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "NAMA_WIFI_ANDA";
6  const char* password = "PASSWORD_WIFI_ANDA";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18   <meta name="viewport" content="width=device-width, initial-scale=1">
19   <title>IoT Dashboard</title>
20   <style>
21     body { font-family: Arial; text-align: center; margin-top: 50px; }
22     button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
23     .btn-on { background-color: #4CAF50; color: white; border: none; }
24     .btn-off { background-color: #f44336; color: white; border: none; }
25     .sensor-box { font-size: 24px; font-weight: bold; }
26   </style>
27 </head>
28 <body>
29   <h1>ESP8266 Web Server</h1>
30   <div class="sensor-box">
31     <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
32   </div>
33   <h2>Kendali Relay</h2>
34   <a href="/relay/on"><button class="btn-on">ON</button></a>
35   <a href="/relay/off"><button class="btn-off">OFF</button></a>
36 </body>
37 </html>
38 )rawliteral";
39
40 void handleRoot() {
41   String html = index_html; // Salin kerangka HTML ke variabel dinamis
42   float t = dht.readTemperature();
43
44   // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
45   if (isnan(t)) {
46     html.replace("%TEMPERATURE%", "--");
47   } else {
48     html.replace("%TEMPERATURE%", String(t));
49   }
50
51   server.send(200, "text/html", html);
52 }
53
54 void handleRelayOn() {
55   digitalWrite(relayPin, HIGH);
56   server.sendHeader("Location", "/");
57   server.send(303);
58 }
59
60 void handleRelayOff() {
61   digitalWrite(relayPin, LOW);
62   server.sendHeader("Location", "/");
63   server.send(303);
64 }
```

```

66 void setup() {
67   Serial.begin(115200);
68   pinMode(relayPin, OUTPUT);
69   digitalWrite(relayPin, LOW);
70   dht.begin();
71
72   WiFi.mode(WIFI_STA);
73   WiFi.begin(ssid, password);
74   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
75   Serial.println("\nIP Address Server Anda: ");
76   Serial.println(WiFi.localIP());
77
78   server.on("/", handleRoot);
79   server.on("/relay/on", handleRelayOn);
80   server.on("/relay/off", handleRelayOff);
81   server.begin();
82 }
83
84 void loop() {
85   server.handleClient();
86 }

```

Buka **Serial Monitor** , dan catat alamat IP yang didapat

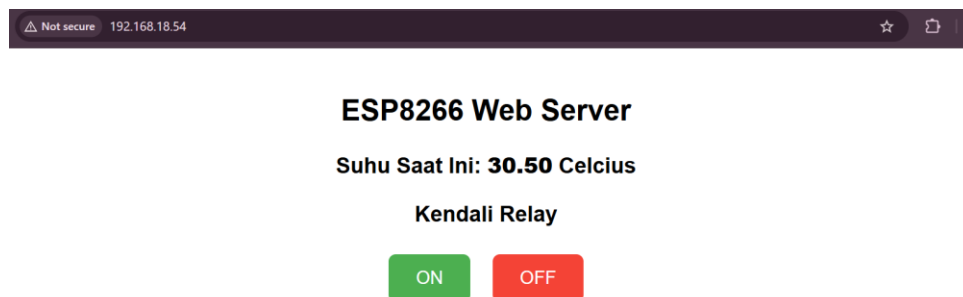
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

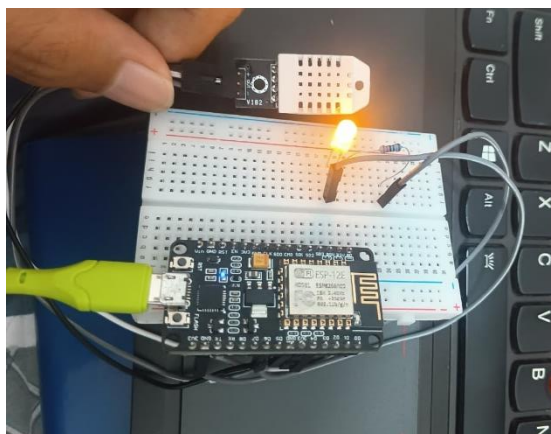
...

IP Address Server Anda:
192.168.18.54

Tampilan WEB local:



WEB menampilkan keterangan suhu dari sensor DHT22 dan Jika button ON/OFF di web ditekan maka LED akan menyala/mati:

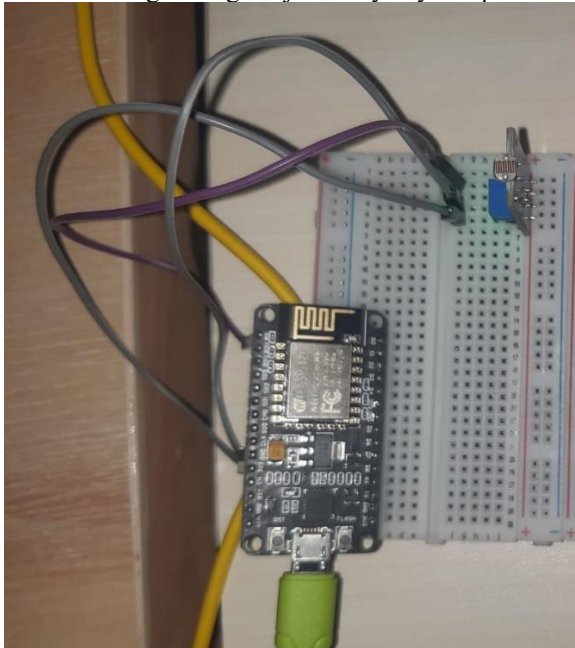


3. NodeMCU sebagai HTTP Client (Meminta Layanan)

Skenario: Anda sekarang memprogram NodeMCU kedua (atau meminta teman kelompok di sebelah Anda) untuk menjadi Client . Perangkat Klien ini dipasang LDR. Jika LDR tertutup kegelapan, ia akan mengirimkan sinyal HTTP secara jarak jauh (melalui jaringan Wi-Fi) untuk menyalakan LED di Server milik teman Anda (Praktikum 1).

Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan lalu menghubungkan jalur sinyalnya ke pin A0 .



Langkah pemrograman:

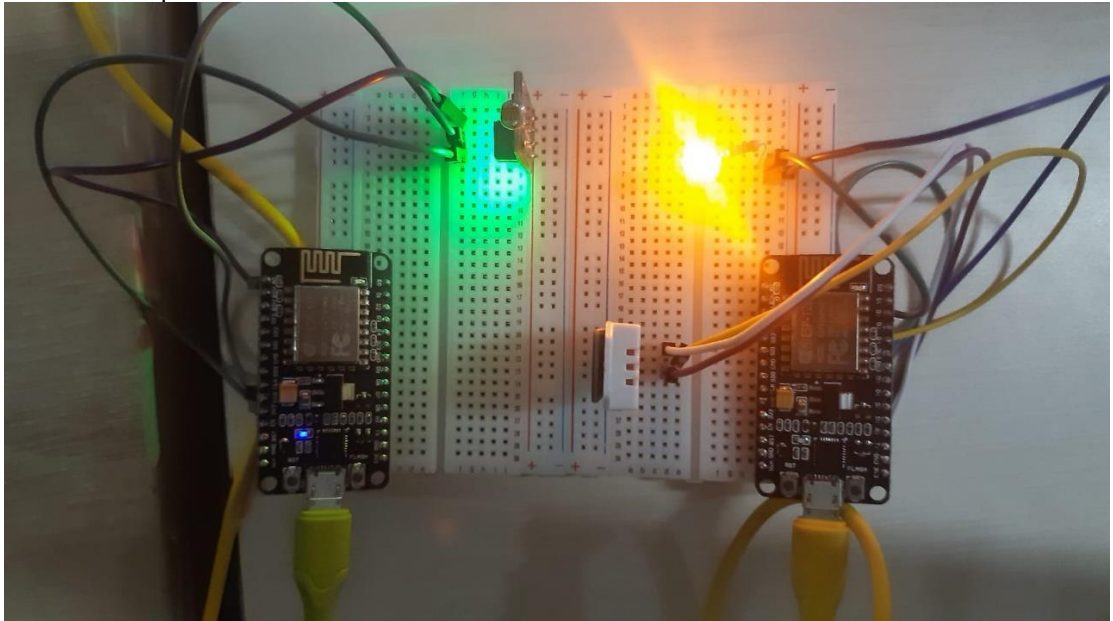
```
praktikum3_5_2_copy_20260921162945.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4
5  const char* ssid = "XP";
6  const char* password = "1a2b3c4d";
7
8  const char* serverName = "http://10.36.43.193/relay/on";
9  const byte ldrPin = A0;
10
11 void setup() {
12     Serial.begin(115200);
13     WiFi.mode(WIFI_STA);
14     WiFi.begin(ssid, password);
15     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16     Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20     int ldrValue = analogRead(ldrPin);
21
22     if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23         WiFiClient client;
24         HTTPClient http;
25
26         http.begin(client, serverName);
27         int httpStatusCode = http.GET();
28
29         Serial.print("HTTP Response code: ");
30         Serial.println(httpStatusCode);
31
32         http.end();
33         delay(10000);
34     }
35     delay(2000);
}
```

Output dari serial:

```
Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

*****
Client Terhubung ke Wi-Fi!
HTTP Response code: -1
HTTP Response code: 303
```

Dan hasil dari praktikum 2:



Jika sensor LDR membaca nilai lebih rendah dari 300 maka LED dari ESP lain akan hidup

LATIHAN

1. Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)* ? karena berada dalam Satu Subnet IP yang Sama, Komunikasi HTTP antara NodeMCU Client dan NodeMCU Server memerlukan alamat IP yang saling terjangkau Mode Station (STA) memungkinkan kedua NodeMCU terhubung ke Wi-Fi Router atau Hotspot lokal yang sama, jika memakai AP maka NodeMCU harus memiliki router sendiri.
2. Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server* :

- Menampilkan Kelembapan

```
<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
```

```
float h = dht.readHumidity();

if (isnan(h)) {
  html.replace("%HUMIDITY%", "--");
} else {
  html.replace("%HUMIDITY%", String(h));
}
```

- Tombol Dinamis Tunggal

```
<h2>Kendali Relay</h2>
%RELAY_BUTTON%

int currentRelayState = digitalRead(relayPin);
String buttonHTML = "";

if (currentRelayState == LOW) {
  // Jika Relay OFF -> Tampilkan tombol hijau "NYALAKAN (ON)"
  buttonHTML = "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>";
} else {
  // Jika Relay ON -> Tampilkan tombol merah "MATIKAN (OFF)"
  buttonHTML = "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>";
}

// Ganti placeholder %RELAY_BUTTON% dengan sintaks tombol yang sesuai
html.replace("%RELAY_BUTTON%", buttonHTML);
```

Hasil dari latihan soal 2:

ESP8266 Web Server

Suhu Saat Ini: **30.00** Celcius

Kelembapan: **75.90** %

Kendali Relay

MATIKAN (OFF)

3. Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda tiba-tiba kehilangan daya (*offline*)? (Silakan cabut daya *Server* dan lihat hasil erornya). Akan keluar output : HTTP Response code: -1. Nilai -1 menandakan konstanta error HTTPC_ERROR_CONNECTION_REFUSED pada pustaka ESP8266HTTPClient. Eror ini terjadi karena perangkat *Client* gagal membangun koneksi awal pada Port 80 target IP *Server*. Karena *Server* mati, tidak ada perangkat di jaringan yang merespons *Client*, sehingga koneksi langsung memicu Timeout / Connection Refused.
4. Anda ingin mendesain *Klien Pendeteksi Suhu*. Modifikasi *blok kondisional* if di Praktikum 2 sehingga *Klien* mengirim permintaan (GET Request) sensor **HANYA JIKA** DHT suhu pembacaan melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya:

```
latihan4.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4  #include <DHT.h>
5
6  const char* ssid = "XP";
7  const char* password = "1a2b3C4D";
8
9  const char* serverName = "http://10.36.43.193/relay/on";
10 const byte dhtPin = 2; // GPIO2 / D4
11 DHT dhtClient(dhtPin, DHT22); // Sesuaikan DHT22
12
13 void setup() {
14     Serial.begin(115200);
15     dhtClient.begin();
16     WiFi.mode(WIFI_STA);
17     WiFi.begin(ssid, password);
18     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
19     Serial.println("\nClient Terhubung ke Wi-Fi!");
20 }
21
22 void loop() {
23     float temp = dhtClient.readTemperature();
24
25     if (isnan(temp)) {
26         Serial.println("Gagal membaca dari sensor DHT!");
27         delay(2000);
28         return;
29     }
30
31     Serial.print("Suhu Terbaca: ");
32     Serial.print(temp);
33     Serial.println(" °C");
34
35     if ((WiFi.status() == WL_CONNECTED) && (temp > 35.0)) {
36         WiFiClient client;
37         HTTPClient http;
38
39         http.begin(client, serverName);
40         int httpResponseCode = http.GET();
41
42         Serial.print("HTTP Response code: ");
43         Serial.println(httpResponseCode);
44
45         http.end();
46         delay(10000);
47     }
48     delay(2000);
49 }
```

TUGAS

1. Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

antarmuka (Antarmuka Pengguna) IoT tidak boleh statistik. Saat ini, Server Web kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa perintah. Kita harus menyelesaikannya.

Skenario Proyek (Web Dashboard): Anda akan bertindak sebagai Full-Stack IoT Developer untuk memutakhirkan kapabilitas Web Server pada Latihan 2 , sambil memberikan batasan arsitektur HTTP tradisional.

Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2)

```
<meta http-equiv="refresh" content="5">
```

Dengan menggunakan modifikasi ini, tampilan web akan ter-refresh secara mandiri.

Instruksi Analisis Arsitektur:

- i. Setelah Auto-Refresh berjalan, coba tekan tombol *ON/OFF* (kendali LED) dari *browser* .
- ii. Apa kelemahan visual/interaksi (*berkedip*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim dirinya sebagai *Real-Time Dashboard* ? Tulis evaluasi logistik Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth* , layar berkedip putih, atau potensi bentrokan jika Anda mengklik tepat di detik ke-5).

Hasil analisis:

1. Efek Visual Layar Berkedip (Screen Flickering):

- Setiap kali timer 5 detik tercapai, browser membuang seluruh struktur DOM (Document Object Model) halaman saat ini dan meminta ulang seluruh dokumen HTML dari server.
- Proses ini menyebabkan efek white flash atau layar berkedip putih sejenak. Hal ini menurunkan kualitas pengalaman pengguna (User Experience / UX) dan sangat tidak ideal untuk sebuah sistem yang diklaim sebagai Real-Time Dashboard.

2. Penggunaan Bandwidth dan Pemrosesan Sumber Daya yang Inefisien:

- Padahal data yang berubah setiap 5 detik hanyalah angka suhu dan kelembapan, namun server harus membangun ulang dan mengirimkan seluruh kode HTML, CSS, serta komponen header HTTP secara berulang-ulang (overhead data).
- Kondisi ini membebani memori RAM dan daya pemrosesan mikrokontroler ESP8266 secara berlebihan.